

Scalable Real-Time E-Commerce Clickstream Analytics

SHARMILA RAMARAJ | X24244066

PARTNER: [INSERT NAME AND STUDENT ID]

MSc Cloud Computing, National College of Ireland | Scalable Cloud Programming | August 2026

Abstract- E-commerce teams need historical context and current behaviour at the same time. This project implements a reproducible AWS Lambda architecture that replays a public REES46 electronics clickstream into Amazon Kinesis, processes recent events with AWS Lambda, stores minute aggregates in DynamoDB, and computes historical five-minute product baselines with PySpark on Amazon EMR. A Python serving layer and responsive web dashboard merge both views to identify unusually trending products and quantify funnel drop-off. From 200,000 source rows, 199,965 valid events were normalised; five S3 copies produced a 999,825-event, approximately 290 MB benchmark. On the same two-worker EMR cluster, the one-partition reference completed in 70 s and the eight-partition run in 58 s, a 1.207x speedup and 17.1% lower duration. A deterministic live test verified Kinesis-to-Lambda processing and produced the expected DynamoDB window rows. The results demonstrate the analytical value of combining recent activity with historical norms while showing that fixed Spark and storage overheads limit speedup on modest inputs.

Keywords- clickstream analytics, Lambda architecture, Amazon Kinesis, AWS Lambda, DynamoDB, PySpark, Amazon EMR, auto-scaling

I. INTRODUCTION

Online stores generate product views, cart actions, removals and purchases continuously. Historical reports can show long-term popularity but may hide a sudden spike, while a live counter can mistake normal popularity for an anomaly. The practical question addressed here is: Which products have unusually high activity during the latest five-minute window, and is the larger current funnel drop-off view-to-cart or cart-to-purchase?

The resulting view supports marketing promotion, stock checking, user-experience investigation and platform operations. It is decision support only: it does not identify customers, change prices or contact users. The main contribution is an end-to-end Python implementation with reproducible input, separate batch and speed paths, an explicit serving merge, and measured parallel execution on AWS Academy infrastructure.

A. Objectives

The objectives were to: (1) build a controlled replay producer; (2) compute historical baselines with distributed PySpark; (3) calculate recent product and funnel metrics from Kinesis; (4) combine current and historical views; (5) configure elastic AWS processing; and (6) compare sequential-reference and parallel execution using the same input and cluster.

II. BACKGROUND AND DESIGN RATIONALE

The Lambda architecture separates a complete, recomputable batch view from a low-latency speed view and merges them for queries [1]. This separation fits the use case: batch computation supplies accurate historical norms, whereas recent counters reveal changes early. Spark provides distributed, fault-tolerant transformations derived from the resilient distributed dataset model [2]. Kinesis and Lambda provide a managed event path in which Lambda polls the stream and invokes the consumer for available records [3].

A single stream-only design would be simpler but would require durable state, replay and historical recomputation to answer the same question. A batch-only design would be accurate but stale. The selected hybrid therefore prioritises both correctness and freshness while remaining small enough for a Learner Lab.

III. USE CASE AND DATA

A. Operational Metrics

For product p in the latest five minutes, recent engagement is weighted so that actions nearer to conversion contribute more:

$$\text{trend_score}(p) = \text{views}(p) + 3 \text{ carts}(p) + 5 \text{ purchases}(p).$$

The batch layer computes the historical mean of the same score over observed five-minute windows. Activity lift is the current score divided by this historical mean; a product is labelled unusually trending when lift is at least 2.0. This rule detects relative change, so a normally popular product is not automatically treated as anomalous.

$$\text{view-to-cart drop-off} = 1 - \text{cart sessions} / \text{view sessions}$$

$$\text{cart-to-purchase drop-off} = 1 - \text{purchase sessions} / \text{cart sessions}.$$

These ratios are window indicators, not proof that a named customer permanently abandoned checkout. A separate cart signal is generated after 15 replay-clock minutes if a session has neither purchased nor removed that product.

B. Dataset Preparation

The REES46 source contains anonymised e-commerce behavioural events and provides a compressed electronics event file [4]. Its fields include event time and type, product/category, brand, price, user and session identifiers. The source contained 885,129 events. The experiment selected the first 200,000 source rows, retained 199,965 valid events, and rejected 35 rows (0.0175%) without a session identifier because session funnels cannot be attributed reliably without it.

Item	Verified value
Source events	885,129
Selected / valid	200,000 / 199,965
Rejected	35 (0.0175%)
Canonical JSONL	58 MB
Benchmark	999,825 events; ~290 MB

IV. SYSTEM ARCHITECTURE

Implemented Lambda Architecture

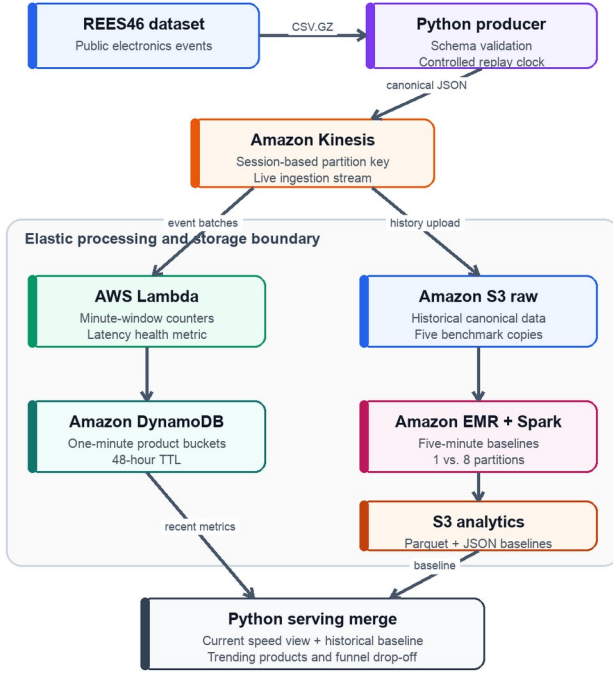


Fig. 1. Implemented batch, speed and serving paths.

The producer normalises the historical CSV.GZ file into canonical JSON, preserves the source timestamp and assigns a replay-clock event time. Session ID is the Kinesis partition key, preserving per-session order within a shard. The live branch invokes a Python Lambda and uses atomic DynamoDB updates for product and health counters in one-minute buckets with time-to-live expiry. The historical branch stores canonical input in S3, where EMR Spark aggregates five-minute windows and writes Parquet plus consolidated JSON baselines.

The serving module reads the recent view and historical product rows, calculates lift, orders products by relative change, and returns a JSON analytical view. A responsive web dashboard presents the same verified values as product activity, a three-stage funnel, service status and batch-performance panels. Its 3D animation is a presentation aid only and does not alter the analytical result. CloudWatch and retained EMR logs provide operational evidence.

V. IMPLEMENTATION

A. Reproducible Ingestion

The Python producer accepts CSV or compressed CSV, maps source event types to the canonical schema, validates timezone-aware timestamps and required identifiers, and supports a record limit, target replay rate and no-sleep benchmark mode. The 199,965-event normalisation completed in 16.804 s, equivalent to approximately 11,900 records/s for local transformation and file output. This figure is not presented as end-to-end Kinesis throughput.

B. Batch Layer

The PySpark job reads JSON from S3, filters invalid rows, groups product events into five-minute windows, pivots event types and calculates activity scores. A second aggregation produces mean views, carts, purchases and trend score per product, while distinct

session counts provide historical funnel context. Outputs are written as compressed Parquet for efficient analytical access and JSON for the serving demonstration.

C. Speed and Serving Layers

The Kinesis event-source mapping invokes the Lambda in batches. For each supported event the function increments the corresponding DynamoDB attribute and a health row containing event count and accumulated processing latency. Partial batch failure identifiers are returned so failed Kinesis records can be retried. The deterministic eight-event fixture produced three rows: two product aggregates and one health row.

The local stateful implementation maintains an exact five-minute deque and session counters for repeatable tests; the AWS Lambda persists equivalent one-minute buckets. The serving merge combines the latest buckets with batch averages, allowing recent counts to be interpreted in context. The approved StreamCart dashboard includes trend and AWS-flow views, pauseable motion, 3D funnel stages and raised performance bars while retaining plain numerical labels for reproducible interpretation.

D. Elasticity and Fault Handling

The EMR 7.13.0 cluster used Spark 3.5.6, one primary node, one core worker and one task worker at launch. Managed scaling was configured for a minimum of two and maximum of four worker instances, with at most one core worker. AWS documents that managed scaling evaluates cluster workload and can increase or decrease core/task capacity [5]. Lambda concurrency can expand with incoming event batches within account and event-source limits. The cluster also used a 20-minute idle-termination control and was manually terminated after measurement.

VI. EXPERIMENTAL METHOD

Correctness was checked first with eight deterministic events covering two products and four sessions. After the event-source mapping reached Enabled, the replay populated DynamoDB and the serving JSON was compared with the known expected funnel and trend values. Seven automated local tests covered models, producer behaviour, speed logic, sequential baseline and serving merge.

For batch performance, five S3-side copies of the 199,965-event canonical subset formed one 999,825-event input of approximately 290 MB. Both runs used the same EMR cluster and program. The reference forced one Spark partition; the parallel case repartitioned by product into eight partitions. Duration was taken from completed EMR step timings. Speedup was calculated as $T(1 \text{ partition}) / T(\text{configuration})$. This controls the data and infrastructure but varies partitions rather than worker count.

VII. RESULTS

A. Real-Time Analytical Answer

In the controlled serving result, product 200 scored 5 against a historical five-minute mean of 2.5. Its activity lift was 2.0, so it was flagged as unusually trending. Product 100 scored 13 against a mean of 13; lift was 1.0 and it was not flagged. Across the four view sessions, three reached cart and one reached purchase. Therefore view-to-cart drop-off was 25%, while cart-to-purchase drop-off was 66.67%. The larger immediate loss was after cart, suggesting that product/checkout friction should be investigated before adding more top-of-funnel promotion.

B. Serving Dashboard

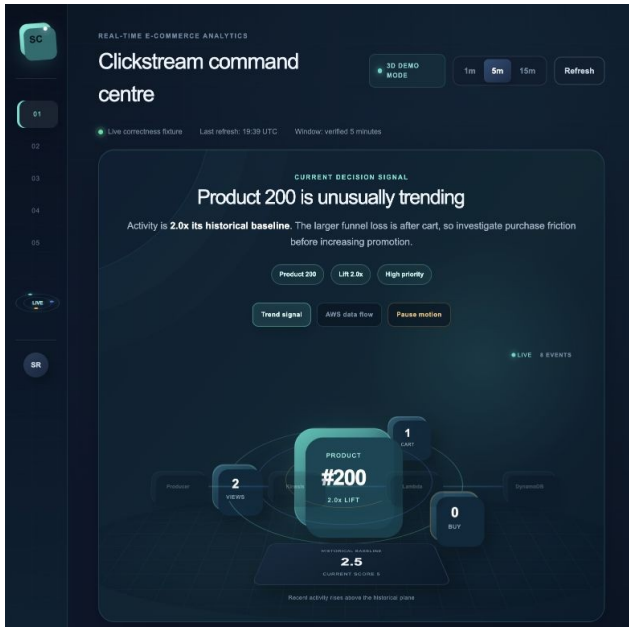


Fig. 2. Approved dashboard showing the verified trend signal.

The dashboard places the real-time answer first: product 200 is shown at 2.0x activity lift, with the current score, baseline and event composition visible in the 3D signal scene. A switchable AWS-flow mode demonstrates how records move from the replay producer through Kinesis and Lambda to DynamoDB. The interface clearly labels the eight-event live fixture separately from the 999,825-event batch benchmark.

C. Batch Performance

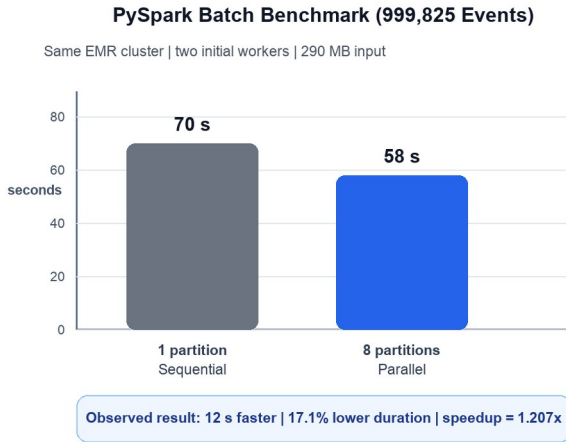


Fig. 3. Same-cluster partitioning comparison on EMR.

Configuration	Part.	Time	Speedup
Sequential reference	1	70 s	1.000x
Parallel execution	8	58 s	1.207x

The parallel run completed 12 s faster, reducing elapsed duration by 17.1%. Output verification found two Parquet part files, one consolidated JSON file, success markers and approximately 9.75 MB of results. The 1.207x gain is useful but well below linear because Spark startup, S3 I/O, JSON parsing, shuffle coordination, task scheduling and final coalescing remain fixed or

partly serial. The modest input also makes distributed overhead a larger fraction of total time.

VIII. CRITICAL ANALYSIS

The strongest aspect is semantic reproducibility: a public dataset, deterministic fixture, explicit formulas and identical benchmark input make the result explainable. The Lambda architecture answers a question that neither layer answers alone. The unusual-trend rule prevents raw popularity from dominating, and the funnel result provides an actionable location for investigation.

The experiment nevertheless has important limits. First, one versus eight partitions on the same two-worker cluster is a parallelism comparison, not a controlled node-scaling experiment; therefore worker efficiency is not claimed. Second, the managed-scaling policy was configured and verified, but the 58-70 s jobs did not maintain demand long enough to prove an observed scale-out. Third, the cloud correctness replay contained only eight events, while the larger dataset evidence applies to preprocessing and EMR batch execution rather than sustained Kinesis load. Fourth, DynamoDB atomic increments are not idempotent, so a successful record retrieved after a partial failure could be counted twice.

Production improvements would add an event ID and deduplication record, emit percentile latency rather than only accumulated latency, retain raw stream records through a managed delivery path, and run repeated trials across several worker counts and data sizes. A longer backlog-generating load test should capture actual scaling timestamps, Kinesis iterator age, Lambda errors/throttles, executor utilisation and cost. Structured Streaming is also a viable future consolidation path because it supports event-time windows, checkpointing and incremental execution [6], although it would change the deliberately separate Lambda speed layer used here.

IX. CONCLUSION

The project implements a compact but genuine AWS clickstream analytics pipeline. Kinesis and Lambda supply fresh counters, S3 and EMR provide recomputable history, DynamoDB stores speed metrics, and a Python serving merge plus dashboard turns both into product-trend and funnel decisions. The verified demonstration identified product 200 as unusually trending and located the larger drop-off between cart and purchase. The EMR test showed a measured 1.207x speedup from partitioned execution, while the analysis explains why the gain is not linear and avoids treating configuration as proof of scaling. The next priority is a longer, repeated load experiment with idempotent stream updates and observed scale events.

REFERENCES

- [1] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Manning, 2015.
- [2] M. Zaharia et al., 'Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,' *Proc. NSDI*, pp. 15-28, 2012. usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia
- [3] Amazon Web Services, 'Tutorial: Using Lambda with Kinesis Data Streams,' AWS Lambda Developer Guide, 2026. docs.aws.amazon.com/lambda/latest/dg/with-kinesis-example.html
- [4] REES46, 'Events in an electronics and home-appliance store,' Public e-commerce datasets, 2026. data.rees46.com
- [5] Amazon Web Services, 'Using managed scaling in Amazon EMR,' Amazon EMR Management Guide, 2026. docs.aws.amazon.com/emr/latest/ManagementGuide/emr-managed-scaling.html
- [6] Apache Software Foundation, 'Structured Streaming Programming Guide, Spark 3.5.6,' 2026. spark.apache.org/docs/3.5.6/structured-streaming-programming-guide.html

Submission links- Repository: github.com/Sharmila-Ramraj/scalable-clickstream-analytics | Video: [INSERT YOUTUBE OR ONEDRIVE URL]